Date :

6 Inheritance

6.1 What is inheritance?

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviours of a parent object. It is an important part of OOPs (object Oriented programming system).

The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

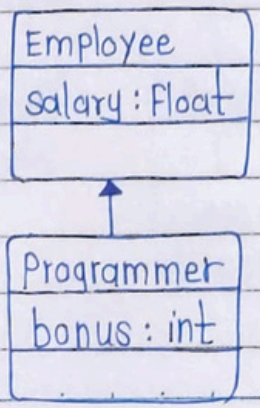
Why use inheritance in Java:

- For method Overriding (so runtime polymorphism can be archieved)
- For code reusability.

Syntax:

```
class Subclass-name extends Superclass-name
{
  // methods and fields
}
```

Example:



Date :

Example :

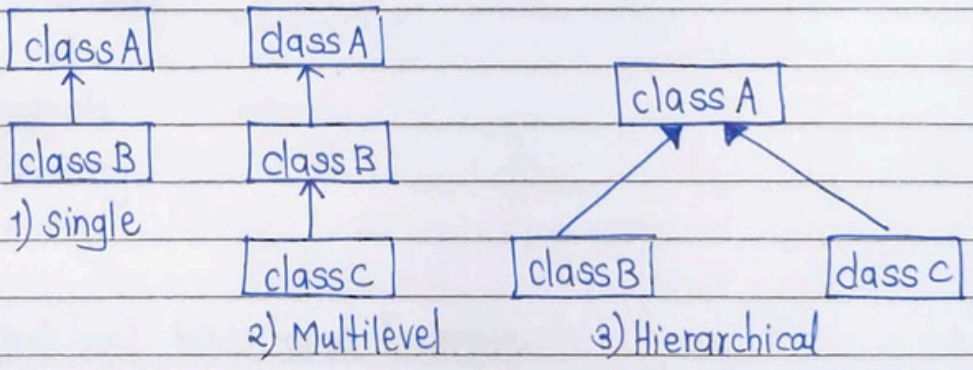
```
class Employee{
  float salary =40000;
}
class Programmer extends Employee{
int bonus =10000;
public static void main(string args[]){
  Programmer P =new Programmer();
   System.out.println("programmer salary is:"+p.salary);
   System.out.println(" Bonus of programmer is:"+p.bonus);
}
}
```

Output:

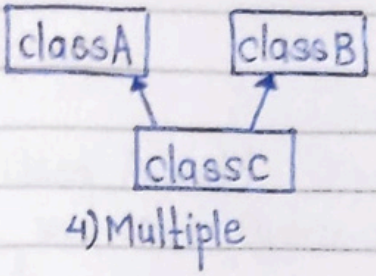
```
programmer salary is : 40000.0
Bonus of programmer is : 10000
```

6.2 Types of inheritance in Java :

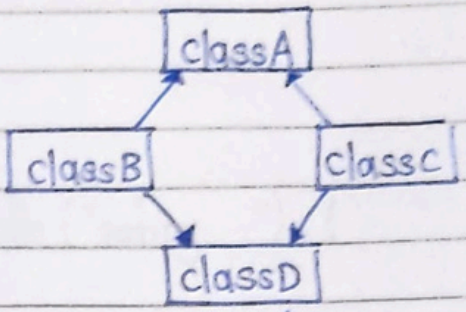
On the basis of class, there can be three types of inheritance in java : single, multilevel and hierarchical. In Java programming, multiple and hybrid inheritance is support through interface only.



Multiple Inheritance is not supported in Java through class



4) Multiple



5) Hybrid

Single Inheritance Example:

When a class inherits another class, it is known as a single inheritance.

```
class Animal{
void eat(){system.out.println("eating...");}
}
class dog extends Animal{
void bark(){system.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

Output:

```
barking....
eating....
```

Multilevel Inheritance Example

When there is a chain of inheritance, it is

Date :

known as multilevel inheritance.

Example:

```
class Animal {
  void eat() { System.out.println("eating...");}
}
class dog extends Animal {
void bark() { system.out.println("barking...");}
}
class BabyDog extends dog {
void sweep() { system.out.println("weeping...");}
}
class inheritance2 {
 public static void main (string args[]){
  BabyDog d = new BabyDog();
   d.weep();
   d.bark();
   d.eat();
 }
}
```

Output:

```
weeping....
barking....
eating....
```

Hierarchical Inheritance Example:

When two or more classes inherits a single class, it is known as hierarchical inheritance. Ex: Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

```
class Animal {
 void eat() { system.out.println("eating...");}
}
class Dog extends Animal{
  void bark() { system.out.println("barking..");}
}
class Cat extends Animal{
  void meow() { System.out.println("meowing..");}
}
class TestInheritance{
  public static void main (string args[]){
  Cat c = new Cat();
   c.meow();
   c.eat();
  }}
```

Output:

```
meowing...
eating...
```

6.3 Super keyword

The 'super' keyword in Java is used to refer to the superclass and its members from within the subclass. It is often used to access overriden methods, constructor chaining, and to differentiate between superclass, subclass members with the same time.

example:

```
class Animal {
   void eat() {
```

Date :

```
    System.out.println("Animal is eating.");
  }
}
class Dog extends Animal{
  void eat(){
    System.out.println("Dog is eating.");
  }
void eatsomething(){
  Super.eat(); //calls the eat() method of the superclass
  }
}
```

6.4 Aggregation in Java

Aggregation is a form of association between classes where one class represents the part of or container (the 'whole' class) while another class represents the part of contained objects. Aggregation is typically expressed as a "has-a" relationship, where one class contains objects of another class as its instance variables.

Example of aggregation:

```
class Address{
   String street;
   String city;
    Address(String street, String city){
     this.street = street;
     this.city = city;
    }
  }
class Person{
```